

```
int main()
{
}
```

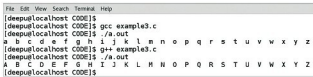
Example 2

There are many such examples where C and C++ show their differences in the least expected places. For example, the program *Example 3* will display different outputs on the screen when compiled as C and C++ programs.

```
#include<stdio.h>
int main()
{
    int i,j;
    i=sizeof('a');
    i=-i/3*32+65;
    for(j=1;j<=26;j++)
    {
        printf("%c ",j);
    }
    printf("\n");
}
```

Example 3

The program when compiled as a C program will output the lower case letters, and the same program will output upper case letters when compiled as a C++ program. This is due to the fact that the size of a character literal is 4 bytes in C and 1 byte in C++, because C treats a character literal as integer type and C++ treats a character literal as character type. Figure 1 shows the output of the program execution as C and as C++.



```
File Edit View Search Terminal Help
[deepu@localhost ~]$ gcc example3.c
[deepu@localhost ~]$ ./a.out
a b c d e f g h i j k l m n o p q r s t u v w x y z
[deepu@localhost ~]$ g++ example3.c
[deepu@localhost ~]$ ./a.out
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
[deepu@localhost ~]$
```

Figure 1: C and C++ outputs

In C, by default, the return type of a function is an integer. But this default to the *int* rule is not valid in C++. You have to mention the return type as *integer* explicitly. Due to this, the program *Example 4* will show an error when compiled as a C++ program.

```
#include<stdio.h>

fun()
{
}
```

```
int main()
{
    fun();
}
```

Example 4

In C, an empty function prototype tells the compiler that the function can have any number of arguments. But in C++, it means the function cannot take any arguments. For example, the function declaration *int fun()*; in C can receive zero or more arguments, whereas in C++, this function has zero arguments. The program *Example 5* shows an error when compiled as a C++ program.

```
#include<stdio.h>
int fun();
int main()
{
    fun(123);
}
int fun(int n)
{
    printf("\n Number is %d\n",n);
}
```

Example 5

The unary increment and decrement operators in C++ return an lvalue. But in C, they return an rvalue. So *(++i)++*; is valid in C++ but it will lead to an error in C. The program *Example 6* will show the error message 'lvalue required' in C whereas in C++, the program will successfully compile and show the output as 'i = 3'.

```
#include<stdio.h>

int main()
{
    int i=1;
    (++i)++;
    printf("ni = %d\n",i);
}
```

Example 6

In C, we cannot get the address of a register variable. But in C++, it is possible to get the address of a register variable. The program *Example 7* is not an error-free C program but when compiled as C++, the program runs without any errors.

```
#include<stdio.h>

int main()
```